

Rapport de Projet

1 Résumé

1.1 Ce qui a été implémenté

Un jeu de la vie basique évoluant dans un terminal, avec une grille torique ou non et la possibilité de choisir entre 3 configurations de base en modifiant le contenu du fichier principal (JeuDeLaVie.java)

1.2 Les prochaines étapes

- Créer une interface graphique
- Commencer à se répartir les différentes tâches présentes sur fonctionnalités.pdf

2. Conception

2.1 Diagramme de classes

[Voir fichier "diagramme#.png pour le diagramme UML]

2.2 Description des classes principales détaillée

Cellule

Représente une cellule individuelle du jeu avec son état (vivant/mort).

- **Attributs :**
 - `actuel` : état actuel de la cellule (boolean)
 - `futur` : état futur calculé de la cellule (boolean)
 - `voisins` : liste des cellules voisines (List<Cellule>)
- **Méthodes principales :**
 - `estVivant()` : vérifie si la cellule est vivante
 - `calculeEtatFutur()` : calcule l'état futur selon les règles du jeu
 - `basculer()` : actualise l'état actuel avec l'état futur calculé

Grille

Gère la grille 2D de cellules et coordonne les interactions.

- **Attributs :**
 - `hauteur`, `largeur` : dimensions de la grille
 - `estTorrique` : indique si les bords de la grille sont connectés

- `grid` : tableau 2D de cellules
- **Méthodes principales :**
 - `affecteVoisins()` : définit les voisins pour chaque cellule
 - `jeu()` : calcule l'état futur pour toutes les cellules
 - `actualise()` : applique les changements d'état calculés
 - `getVoisins(i, j)` : trouve les voisins d'une cellule aux coordonnées (i, j)

Configurations

Contient des méthodes statiques pour créer différentes configurations initiales.

- **Méthodes :**
 - `gAlea()` : génère une configuration aléatoire
 - `gPlanneur()` : génère une configuration avec un planneur
 - `gCannon()` : génère une configuration avec un canon à planeurs

JeuDeLaVie

Classe principale qui gère l'exécution du jeu.

- **Méthodes :**
 - `main()` : point d'entrée du programme
 - `clearScreen()` : efface l'écran pour l'animation

3. Guide de développement

Compilation :

```
javac *.java
```

Création du JAR :

```
jar cvfm source1.jar manifest.txt *.class
```

Le fichier manifest.txt doit contenir:

```
Main-Class: JeuDeLaVie
Class-Path: .
```

Exécution :

```
java -jar source1.jar
```